RapCluster

User guide with worked examples

Contents

1 Overview **3**

2 Uploading data **4**

2.1 Example datasets 4

3 Preprocessing **5**

3.1 Remove rows with more than … % missing values 5

3.2 Fill remaining missing values with 5

3.3 Transformation 6

3.4 Remove all-zero rows 7

3.5 Remove low-variance columns 7

3.6 Remove highly correlated columns 7

3.7 Scaling 8

3.8 The running example after all steps 9

4 Exploring the data before clustering **9**

4.1 How outliers are found 10

5 Dimensionality reduction **10**

6 Clustering algorithms **11**

6.1 Centre-based: KMeans, MiniBatchKMeans, BisectingKMeans 12

6.2 Hierarchical: AgglomerativeClustering and BIRCH 13

6.3 SpectralClustering 13

6.4 Mixture models: GaussianMixture and BayesianGaussianMixture 13

6.5 Density-based: DBSCAN, HDBSCAN, OPTICS 14

6.6 MeanShift 14

6.7 AffinityPropagation 14

7 Checking and running **15**

7.1 Preflight Check 15

7.2 Running 16

8 Results **16**

8.1 Metrics 16

8.2 Visualization tabs 17

8.3 Method Details and Cluster Details 17

9 Characterization **18**

9.1 Cluster summary 18

9.2 Cluster profiles: median with interquartile range 18

9.3 Features that distinguish a cluster 18

9.4 Feature distribution by cluster 19

9.5 Differences between clusters per feature (Kruskal–Wallis) 19

9.6 Outliers and noise 19

10 Stability **20**

10.1 Cluster stability (Jaccard) 20

10.2 Agreement between solutions (ARI and NMI) 20

10.3 Row stability and consensus 21

11 Working with many runs **21**

11.1 Run History 21

11.2 Run Leaderboard 21

11.3 Compare Runs 22

11.4 Parameter Sweep 22

12 Downloads and reproducing a run **23**

13 Running RapCluster yourself **23**

Glossary **24**

1 Overview

A typical session looks like this:

1. Upload a table, or load one of the example datasets.
2. Choose the preprocessing (how missing values are handled, whether to take a log, how to scale).
3. Optionally reduce the number of dimensions (PCA, UMAP or t-SNE).
4. Choose a clustering algorithm and its parameters.
5. Run, look at the results, try other settings, compare the runs and download what you need.

Every run is stored in the session. You upload the data once and can then try as many settings as you like; the runs appear in **Run History** and can be compared with each other. All data and runs are deleted when you close the page.

Most labels on the page have a small **?** next to them. Hover over it (or tap it, or reach it with the Tab key) to see a short explanation. This guide is the long version of those explanations.

The running example

Many examples in this guide use the same small table: five proteins measured in four fractions of a separation. The numbers are intensities, and empty cells are missing values.

Name	F1	F2	F3	F4
ProtA	10	50	100	20
ProtB	1	5	10	2
ProtC	100	20	10	50
ProtD	0	0	0	0
ProtE	40			30

Look at ProtA and ProtB: both peak in F3, but ProtA is ten times more abundant. ProtC peaks in F1. ProtD has no signal at all, and ProtE is half empty. We will follow these five rows through the whole pipeline.

2 Uploading data

Field	What it does
Select File	CSV, TSV, XLSX or XLS, up to 50 MB. One row per item (protein, gene, sample, ...), one column per measurement.
Or load example data	Loads one of the bundled datasets and fills in the name column, the data start column and suggested preprocessing.
Sheet	Excel only: the worksheet that holds the data.
Name of Node Column	The column that identifies each row. It labels points, tables and downloads, but it is not used for clustering. If the column does not exist, rows are called `Node_1`, `Node_2`, ...
Data Start Column Index	Position of the first numeric column, counting from 0. This column and every column after it are clustered. Columns before it are ignored.

Example: the data start column

Your file has these columns:

0	1	2	3	4	...
Name	Gene	Class	F1	F2	...

The measurements start at F1, which is position 3 (counting from 0), so you set the data start column index to **3** and the name column to **Name**. Gene and Class are kept in the file but not clustered. If you entered 2 by mistake, the text column Class would be read as data; every cell in it would become a missing value, and the Preflight Check would warn you about non-numeric values.

Numbers with a decimal comma (`1,5`) are accepted and read as `1.5`. Any cell in a data column that is not a number becomes a missing value.

2.1 Example datasets

Example	Size	Useful for
Protein profiles	3000×120	The original RapCluster use case: fractionation profiles.
Time-course profiles	1200×24	Six expression patterns over time (rising, falling, late peak, ...).
Gaussian blobs	600×10	Five well-separated groups. A sanity check: almost every method should find them.
Uneven density with noise	700×8	Groups of different density plus scattered noise. Made for DBSCAN, HDBSCAN and OPTICS.
Iris	150×4	The classic small dataset with three flower species.

Each example has a `Class` column with the true groups. It sits before the data start column, so it is not used for clustering, but you can compare your result against it.

3 Preprocessing

Raw measurements are rarely ready for clustering. Some cells are empty, a few very large values dominate everything else, and rows or columns are on different scales. Preprocessing deals with this. The steps always run in this order:

1. Read the data columns as numbers.
2. Remove rows with too many missing values.
3. Fill the remaining missing values.
4. Transform the values (log or none).
5. Remove rows that are all zero.
6. Remove low-variance and highly correlated columns (both optional).
7. Scale the rows or the columns.

The defaults reproduce the original RapCluster pipeline: keep every row, fill missing values with 0, take $\log_2(x + 1)$, remove all-zero rows, and scale each row to a z-score. The settings are saved with every run, so a run can always be repeated exactly.

3.1 Remove rows with more than ... % missing values

For each row, RapCluster counts the share of its data columns that are empty (or not a number). If that share is *above* the value you enter, the row is removed. Exactly the threshold is still allowed.

Example: threshold 25 %

With four data columns, each missing cell is 25 %.

Row	Missing cells	Share	Result
ProtA	0 of 4	0 %	kept
ProtD	0 of 4	0 %	kept (zeros are values, not missing)
ProtE	2 of 4	50 %	**removed**

A row with one empty cell (25 %) would still be kept.

- **100** (the default) keeps every row.
- **0** keeps only complete rows.
- A row that is mostly empty ends up made mostly of filled-in values, and its cluster then says more about the fill method than about the data. Values between 25 and 50 % are a common compromise.

3.2 Fill remaining missing values with

The rows that survive the filter can still have empty cells. These are filled per column with one of:

Option	Meaning
0 (default)	Missing means “not there”.
Column minimum	The smallest observed value of that column. Missing means “below the detection limit”.
Half of the column minimum	As above, but a little lower than anything that was measured.
Column median	A neutral middle value. It does not push the row up or down, but it can blur real differences.

Example: one column, four ways

A column has the values 50, 5, 20 and one empty cell.

Option	Filled value
0	0
Column minimum	5
Half of the column minimum	2.5
Column median	20

In proteomics a missing intensity usually means the protein was too low to be detected, not that it is absent, so the minimum or half the minimum is often more realistic than 0. Note that zeros count as observed values: if a column contains a 0, its minimum is 0, and “column minimum” then gives the same result as “0”.

A column without any observed value is filled with 0, and the Preflight Check lists it.

3.3 Transformation

Option	Formula and use
$\log_2(x+1)$ (default)	Compresses large values. The +1 keeps 0 at 0.
$\log_{10}(x+1)$	The same with base 10. The shape is identical; only the numbers are smaller.
Signed log	$\text{sign}(x) \cdot \log_2(\lvert x \rvert + 1)$. For data with negative values, such as log ratios or differences.
None	The values are used as they are.

Intensities often span several orders of magnitude. Without a log, one protein with an intensity of 1,000,000 outweighs everything else, and the clustering is mostly about which proteins are abundant. After the log, a step from 10 to 100 counts about as much as a step from 1,000 to 10,000.

Example: what the log does

x	0	1	3	7	1023
$\log_2(x+1)$	0	1	2	3	10

A value 1000 times larger becomes only about 10 units larger. For negative data the signed log keeps the sign: -8 becomes -3.17 and $+8$ becomes $+3.17$.

$\log(x+1)$ is not defined for values of -1 or less. If your data has such values, RapCluster stops with a message and asks you to choose the signed log or no transformation. The Preflight Check finds these values before you run.

3.4 Remove all-zero rows

On by default. A row in which every value is 0 after filling has no profile: it cannot be scaled (all its values are equal) and it would form a meaningless cluster of its own.

Example

ProtD is 0 in every fraction and is removed. A row that was *completely empty* also ends up here when missing values are filled with 0: after filling it is all zeros, so it is removed too.

3.5 Remove low-variance columns

Optional. A column is removed when its standard deviation (after the transformation) is at or below the threshold. With the threshold at 0, only constant columns are removed.

Example

A fraction in which every protein has the value 5 has a standard deviation of 0. It carries no information that could separate rows, so it is removed. With a threshold of 0.1, a column whose values vary only between 4.98 and 5.02 would be removed as well.

3.6 Remove highly correlated columns

Optional, threshold 0.95 by default (allowed: 0.5 to 1). The columns are checked from left to right. A column is removed when its correlation with an earlier column that was kept is above the threshold (in absolute value). The earlier column always stays.

Example

Three columns over four rows:

	C1	C2	C3
Row 1	1	2	5
Row 2	2	4	3
Row 3	3	6	4
Row 4	4	8	1

C2 is exactly twice C1, so their correlation is 1.00 and C2 is removed ("correlated with C1"). C3 has a correlation of -0.83 with C1, which is below 0.95 in absolute value, so it stays.

Note

Do not use this filter for profile data. Neighbouring fractions or time points are naturally correlated, and the filter would throw away much of the profile. It is meant for tables whose columns are different measurements, some of which repeat each other.

3.7 Scaling

Clustering groups rows by how close their values are. Without scaling, whatever has the largest numbers dominates those distances. Scaling makes the values comparable. The main choice is whether to scale each *row* or each *column*.

Option	What it does
Per-row z-score (default)	Each row gets mean 0 and standard deviation 1: $z = (x - \text{mean})/\text{sd}$.
Per-row min–max	Each row's smallest value becomes 0 and its largest becomes 1.
Per-column z-score	Each column gets mean 0 and standard deviation 1.
Per-column robust	Like the z-score, but with the median and the interquartile range, so outliers have less influence.
Per-column min–max	Each column is squeezed to the range 0 to 1. One extreme value can squash all the others.
None	The transformed values are clustered directly.

Per-row scaling compares the shape of each row. It ignores the overall level, so it suits ordered measurements of the same quantity: fractions, time points, a gradient.

Example: per-row scaling

ProtA, ProtB and ProtC after $\log_2(x+1)$, then scaled per row:

	after $\log_2(x+1)$				per-row z-score			
	F1	F2	F3	F4	F1	F2	F3	F4
ProtA	3.46	5.67	6.66	4.39	−1.30	0.51	1.32	−0.54
ProtB	1.00	2.58	3.46	1.58	−1.23	0.45	1.38	−0.61
ProtC	6.66	4.39	3.46	5.67	1.32	−0.54	−1.30	0.51

Before scaling, ProtA is closer to ProtC than to ProtB, simply because both have large numbers. After scaling, ProtA and ProtB are almost identical (same peak in F3) and end up in the same cluster, while ProtC (peak in F1) is far away. That is usually what you want for profiles.
With per-row min–max the picture is the same: ProtA becomes 0, 0.69, 1, 0.29 and ProtB 0, 0.64, 1, 0.24.

Per-column scaling puts different measurements on the same scale. Use it when the columns measure different things.

Example: per-column scaling, iris

In the iris data, sepal length lies between about 4 and 8 cm, petal width between 0.1 and 2.5 cm. Without scaling, a difference of 1 cm counts the same in both columns, although 1 cm is a small step for sepal length and a huge one for petal width. After a per-column z-score, every column has mean 0 and standard deviation 1, and each measurement counts equally. Per-row scaling would make no sense here: it would compare sepal length with petal width inside one flower.

Rule of thumb: same kind of measurement in every column and you care about the pattern (proteomics profiles, time courses): per row. Columns are different kinds of measurement: per column.

The Preflight Check helps here. It warns when you scale per row with fewer than 3 columns (almost nothing is left of the data), gives a hint with 10 columns or fewer, and warns when you use no scaling while the column spreads differ by more than a factor of 100.

3.8 The running example after all steps

With the default settings (keep all rows, fill with 0, $\log_2(x+1)$, remove all-zero rows, per-row z-score):

	F1	F2	F3	F4	What happened
ProtA	−1.30	0.51	1.32	−0.54	
ProtB	−1.23	0.45	1.38	−0.61	same shape as ProtA
ProtC	1.32	−0.54	−1.30	0.51	mirror image of ProtA
ProtD		removed			all zero
ProtE	1.08	−1.00	−1.00	0.92	two cells filled with 0

Notice ProtE. Its two missing cells became 0, so after scaling it looks like a protein with a clear dip in F2 and F3. That dip comes only from the fill value. With the missing filter at 25 %, ProtE would have been removed instead. This is exactly the kind of decision preprocessing is about.

4 Exploring the data before clustering

Explore data with these settings shows the data after your preprocessing, without clustering. It is the place to check whether the settings do what you expect. The panel has five tabs.

Tab	What it shows
Missing values	How many rows have no, few or many missing values (none, ≤10 %, 10–25 %, . . .), and the missing share per column.
PCA variance	How much of the variation the first principal components explain. If two components explain 90 %, a 2D plot shows most of the structure. If they explain 20 %, it hides most of it.
Distributions	One box plot per column, before and after scaling.
Correlation	A heatmap of the correlations between the columns (up to 150 columns).
Outliers	The rows that lie furthest from the typical row.

Note

In the Distributions tab, “before scaling” is not the raw data. Missing values are already filled and the transformation is already applied; only the scaling is left out. The tab therefore shows what the scaling step does.

4.1 How outliers are found

1. RapCluster builds the *median profile*: for each column, the median over all rows. This is the "typical row".
2. For each row it measures the Euclidean distance to this median profile.
3. It turns the distances into *robust z-scores*:

$$z = \frac{d - \text{median}(d)}{1.4826 \cdot \text{MAD}(d)},$$

   where MAD is the median absolute deviation. The factor 1.4826 makes the MAD comparable to a standard deviation for normally distributed data.
4. The 25 rows with the largest z are listed, together with a histogram of all distances.

Example: robust z-score

Seven rows have these distances to the median profile:

$$1.0,\ 1.1,\ 1.2,\ 0.9,\ 1.0,\ 0.8,\ 5.0$$

The median distance is 1.0. The absolute deviations from it are $0, 0.1, 0.2, 0.1, 0, 0.2, 4.0$, with median 0.1, so the scaled MAD is $1.4826 \times 0.1 = 0.148$. The last row gets

$$z = \frac{5.0 - 1.0}{0.148} \approx 27,$$

while all others lie between -1.35 and 1.35. A robust z-score above about 3.5 is unusual and worth a look.
Why not the ordinary mean and standard deviation? The outlier itself would raise both, and its z-score would only be about 2.4. Medians are hardly affected by a single extreme value.

The outliers depend on your settings. With per-row scaling, an outlier is a row with an unusual *shape*, not one that is merely very high or low. To find rows with extreme values, look with per-column scaling or none. RapCluster never removes outliers on its own. If a row looks like an error, remove it from your file and upload again.

5 Dimensionality reduction

Optional. A reduction maps the data to fewer dimensions before clustering. Two things are important:

- The clustering uses **all** chosen dimensions (`n_components`). The plots show the first two.
- With **None**, the clustering uses all preprocessed columns. The scatter plot then shows a PCA projection, for display only.

Method	Idea	Main parameters (default)
None	Cluster on all features.	–
PCA	Linear projection onto the directions of largest variance. Fast and deterministic.	`n_components` (2), `svd_solver` (auto)
UMAP	Non-linear. Keeps neighbours together and much of the global layout. Good input for density-based clustering.	`n_components` (2), `n_neighbors` (15), `min_dist` (0.1), `metric` (euclidean)
t-SNE	Non-linear. Keeps close neighbours together and gives clearly separated groups, but sizes and distances in the plot mean little. 2 or 3 dimensions, slow on large data.	`perplexity` (30), `learning_rate` (200), `n_iter` (1000)

All three have a `random_state` (default 42). The same seed gives the same result.

Example: when to reduce

The protein profiles example has 120 fractions. Clustering on all 120 is fine for KMeans, but density-based methods such as HDBSCAN struggle in many dimensions, because in high dimensions all distances look alike. A common approach is UMAP to 5–10 dimensions, then HDBSCAN on those. With `n_neighbors` = 5, UMAP focuses on very local structure and may split real groups; with 100 it looks at the big picture and may merge small groups.
For iris, with only 4 columns, a reduction is not needed: cluster on all columns and use the plot only for display.

Note

Do not cluster on a t-SNE result unless you know why. t-SNE exaggerates gaps between groups and does not keep distances, so clusters found in a t-SNE map can be artefacts of the map. Use it to look at the data; use PCA or UMAP as input for clustering.

6 Clustering algorithms

RapCluster offers 13 algorithms. They differ in what they assume a cluster is: a round group around a centre, a dense region, a branch of a tree, or a component of a probability mixture. No algorithm is best for all data.

With **Use Default Parameters** ticked, the algorithm runs with the defaults listed below. Untick it to change single parameters. Each parameter has a **?** with its meaning, default and allowed range, and **Learn More** opens a longer guide.

Algorithm	Idea	Noise	You mainly set
KMeans	Rows close to the mean of their group	no	`n_clusters`
MiniBatchKMeans	KMeans on small random batches; for large data	no	`n_clusters`
BisectingKMeans	Start with one cluster, keep splitting one in two	no	`n_clusters`
Agglomerative	Merge the closest clusters step by step	no	`n_clusters`, `linkage`
BIRCH	Compress the data into a tree of subclusters; for large data	no	`threshold`, `n_clusters`
Spectral	Cluster a similarity graph; finds curved shapes	no	`n_clusters`, `affinity`
GaussianMixture	Mixture of Gaussian distributions	no	`n_components`
BayesianGaussianMixture	Mixture that leaves unneeded components empty	no	maximum `n_components`
DBSCAN	Connected dense regions	yes	`eps`, `min_samples`
HDBSCAN	DBSCAN over all density levels	yes	`min_cluster_size`
OPTICS	Density ordering, cut at steep changes	yes	`xi`, `min_samples`
MeanShift	Every density peak is a cluster	optional	`bandwidth`
AffinityPropagation	Rows elect exemplars; small data only	no	`preference`

Noise. Density-based algorithms do not force every row into a cluster. Rows in sparse regions get the label -1 ("noise") and are shown in grey. Noise is not a cluster: it is left out of the cluster count and the quality metrics.

6.1 Centre-based: KMeans, MiniBatchKMeans, BisectingKMeans

KMeans places `n_clusters` centres (default 8) and assigns every row to the nearest one. It then moves each centre to the mean of its rows and repeats until nothing changes. It works best for round clusters of similar size, and you have to know (or try) the number of clusters.

- **MiniBatchKMeans** updates the centres from small random batches instead of all rows. It is much faster on large data and slightly less exact.
- **BisectingKMeans** starts with all rows in one cluster and splits a cluster in two (with 2-means) until there are `n_clusters`. With `bisecting_strategy` = `biggest_inertia` (default) it splits the cluster with the largest spread, with `largest_cluster` the one with the most rows.

Example

Load *Gaussian blobs*, keep the defaults for preprocessing except scaling (per-column z-score) and run KMeans with `n_clusters` = 5. It finds the five blobs almost perfectly. With `n_clusters` = 8 (the default) it still runs, but splits some blobs into pieces: KMeans always returns exactly the number you ask for.

6.2 Hierarchical: AgglomerativeClustering and BIRCH

Agglomerative clustering starts with every row as its own cluster and merges the two closest clusters, again and again, until `n_clusters` are left (default 8). "Closest" depends on the `linkage`:

Linkage	Distance between two clusters
ward (default)	The merge that increases the within-cluster spread the least. Gives compact clusters. Needs the euclidean metric.
complete	The largest distance between any two of their rows.
average	The average distance between their rows.
single	The smallest distance. Can follow long chains, which is useful for elongated shapes but often produces one huge cluster.

BIRCH reads the data once and builds a compact tree of small subclusters, each covering rows within `threshold` (default 0.5) of each other. Those subclusters are then grouped into `n_clusters`. With `n_clusters` set to None, the subclusters themselves are the result. BIRCH is one of the two methods (with MiniBatchKMeans) suited to very large data.

6.3 SpectralClustering

Builds a graph in which each row is connected to its `n_neighbors` nearest rows (default 10, affinity `nearest_neighbors`) and splits this graph into `n_clusters` well-connected parts. Because it follows the graph, it can find curved or ring-shaped clusters that KMeans cannot. The affinity `rbf` uses a full $n \times n$ similarity matrix and needs a lot of memory.

6.4 Mixture models: GaussianMixture and BayesianGaussianMixture

A Gaussian mixture assumes the data was generated by `n_components` Gaussian distributions (default 8) and estimates their means, shapes and weights. Every row gets a *probability* for each component and is assigned to the most likely one. That probability is shown as the row's membership.

`covariance_type` sets the shape of each component:

full (default)	every component has its own shape and orientation
tied	all components share one shape
diag	axis-parallel ellipses
spherical	round, like KMeans

BayesianGaussianMixture takes `n_components` (default 10) only as a *maximum*. It works out how many components it needs and gives the others almost no weight. The result reports how many components were used. A smaller `weight_concentration_prior` favours fewer components; empty (None) means $1/n_{\text{components}}$.

Example

The time-course example has six true patterns in 24 time points. With the defaults (maximum 10, full covariance), BayesianGaussianMixture uses all 10 components and splits some patterns (ARI 0.81 against the true classes). The reason is the full covariance: each component has its own 24×24 covariance matrix, and with 1,200 rows the model can afford to fit small, oddly shaped components. With `covariance_type` = `diag` it uses exactly 6 components and recovers the six patterns perfectly (ARI 1.00). If a run reports that it used every allowed component, try a simpler covariance type or a larger maximum.
To choose n for GaussianMixture, run a parameter sweep over `n_components` and look for the lowest BIC.

6.5 Density-based: DBSCAN, HDBSCAN, OPTICS

These algorithms define a cluster as a dense region and call rows in sparse regions noise. You do not set the number of clusters.

DBSCAN. A row is a *core* row if at least `min_samples` rows (default 5, counting itself) lie within distance `eps` (default 0.5). Core rows that are within `eps` of each other form one cluster, and rows near a core row join it. Everything else is noise.

Example: the effect of eps

On the *uneven density with noise* example: with a very small `eps`, almost every row is noise. With a very large one, all groups merge into one cluster. Somewhere in between DBSCAN finds the groups, but because one `eps` has to fit all of them, it often finds the dense groups and loses the sparse ones to noise, or the other way round. The right `eps` depends on the scale of the data after preprocessing and reduction, so a value that works with z-scores will not work with raw intensities.

HDBSCAN runs DBSCAN for all values of `eps` at once and keeps the clusters that persist longest. It handles clusters of different density. Its main parameter is `min_cluster_size`, the smallest group that counts as a cluster. The defaults on the form are `min_cluster_size` = 2, `min_samples` = 15 and `max_cluster_size` = 20 (empty means no limit); `cluster_selection_method` `eom` prefers a few large clusters, `leaf` many small ones. HDBSCAN also gives a membership probability per row and a *persistence* per cluster (how long the cluster survived; higher is more convincing).

OPTICS orders the rows by density and cuts clusters where the density changes steeply. `xi` (default 0.05) sets how steep a change must be; `min_cluster_size` (default 0.05) is a share of all rows, so 0.05 means at least 5 % of the rows.

6.6 MeanShift

Moves a window of radius `bandwidth` (default 1.0) uphill towards the densest point, starting from many seeds. Every density peak it reaches becomes a cluster. With `cluster_all` on (default), every row is assigned to the nearest peak; with it off, rows far from every peak become noise. A small bandwidth gives many small clusters, a large one few big clusters.

6.7 AffinityPropagation

Rows exchange messages until some of them are elected as *exemplars*, and every row joins its exemplar. The number of clusters follows from `preference` (default -50): a lower (more

negative) value gives fewer clusters. `damping` (0.5 to 1) slows the updates down when the method does not converge. Time and memory grow with the square of the number of rows, so it is only practical for a few thousand rows.

7 Checking and running

7.1 Preflight Check

Check data & settings inspects the data and the chosen methods without clustering and opens the **Preflight Check** panel. **Run Clustering** runs the same check first.

Data checks. Rows and columns used, missing and non-numeric values, values that the log cannot handle (≤ -1), constant columns, all-zero rows, rows with many missing values, strongly correlated columns, duplicate rows, duplicate names, whether the name column exists, the scaling advice described above, the upload size and the estimated size of the result.

Capacity guidance. Every algorithm, reduction method and metric is rated for the number of rows used:

Algorithm	Recommended up to	May be slow up to
MiniBatchKMeans, BIRCH	200,000	1,000,000
KMeans, BisectingKMeans	100,000	500,000
GaussianMixture	50,000	200,000
BayesianGaussianMixture	20,000	100,000
DBSCAN, HDBSCAN	20,000	100,000
AgglomerativeClustering	10,000	30,000
OPTICS	5,000	30,000
MeanShift, SpectralClustering	5,000	20,000
SpectralClustering with rbf	2,000	8,000
AffinityPropagation	2,000	5,000

Above the second number the rating is *Not recommended on the hosted server.* Gaussian mixtures with full covariance on more than 50 dimensions are rated one level worse. For AffinityPropagation and SpectralClustering with rbf, the panel also estimates the memory of the $n \times n$ matrices: $n^2 \times 8$ bytes per matrix (three for AffinityPropagation, one for rbf).

Example: a dataset with 12,000 rows

KMeans	Recommended ($12{,}000 \leq 100{,}000$)
HDBSCAN	Recommended ($12{,}000 \leq 20{,}000$)
AgglomerativeClustering	May be slow (between 10,000 and 30,000)
AffinityPropagation	Not recommended (above 5,000)

For AffinityPropagation the memory estimate is $3 \times 12{,}000^2 \times 8$ bytes $\approx 3{,}300$ MB, about a third of the 10 GB limit of a run. At 20,000 rows it would be about 9,200 MB, almost the whole limit.

How reliable are these numbers?

The limits are estimates, chosen from how each algorithm scales with the number of rows (linear, faster than linear, quadratic) and rounded to fit 10 minutes and 4 GB per run, the limits the server had when they were set. The memory limit is now 10 GB, so for memory they are on the cautious side. They were not measured on the server, and apart from the memory estimate they ignore the number of columns and the parameter values (DBSCAN with a very large `eps`, for example, can be much slower). Treat them as a guide.

7.2 Running

The ratings are recommendations, not rules. If everything is *Recommended*, the run starts at once. Otherwise the panel asks first, with a **Run anyway** and a **Cancel** button. The same happens when the data has problems that will likely make the run fail. **Close** hides the panel.

A run is a background job with a progress bar showing the current stage, and it can be cancelled. What actually stops a run are the limits of the server: 50 MB per upload, 10 minutes and 10 GB per run, and at most 8 runs at the same time (a further run gets "The server is already running 8 analyses"). A run that hits a limit is stopped with a message that says which limit was reached and what to change, for example "Use a faster algorithm, reduce the dimensions with PCA or UMAP, or use a smaller dataset."

8 Results

8.1 Metrics

Clustering has no ground truth, so the metrics measure how well separated and compact the clusters are. They are useful for comparing runs on the *same* data, not as absolute grades. Noise rows are left out.

Metric	Meaning	Better
Clusters found	Number of clusters, without noise.	–
Silhouette	How much closer rows are to their own cluster than to the next one, from −1 to 1. Computed on up to 5,000 rows.	higher
Calinski–Harabasz	Spread between clusters relative to spread within them. No fixed scale.	higher
Davies–Bouldin	How similar each cluster is to its most similar neighbour. 0 is best.	lower
DBCV	Density-based validation, from −1 to 1. Fair to clusters that are not round.	higher
BIC / AIC	Fit of a mixture model with a penalty for complexity (GaussianMixture).	lower
Mean membership	Average probability with which rows belong to their cluster (HDBSCAN, mixtures).	higher
Components used	Components that received rows (BayesianGaussianMixture).	–

Example: silhouette of one row

A row has an average distance of $a = 1.0$ to the other rows of its cluster and $b = 3.0$ to the rows of the nearest other cluster. Its silhouette is

$$s = \frac{b - a}{\max(a, b)} = \frac{3.0 - 1.0}{3.0} = 0.67.$$

A value near 1 means the row clearly belongs where it is, near 0 that it sits between two clusters, and a negative value that it is closer to another cluster. The silhouette score of a run is the average over all rows. As a rough guide, above 0.5 is a clear structure and below 0.25 a weak one.

Note

The silhouette and Calinski–Harabasz favour round, compact clusters. A density-based run that finds two half-moons correctly can score worse than a KMeans run that cuts them in half. For density-based results, look at DBCV.

8.2 Visualization tabs

Scatter Plot
Every row as a point, coloured by cluster. Hover for the name, drag to zoom. For large data the plots show a sample of up to 5,000 rows that includes every cluster; sizes, tables and downloads always use all rows. A caption template below the plot describes the settings for a figure legend.

Clusters Network
One node per cluster, placed at its centre. Edges are thicker and darker the closer two clusters are. Click a cluster to open its sub-network.

Sub-network View
The rows of one cluster, each connected to its nearest neighbours (3 to 20). Draw a lasso around points to plot their raw profiles (as $\log_{10}$ of the filled values).

Heatmap
The preprocessed values as colours, either one summary row per cluster (median or mean) or the rows of one cluster (up to 1,000). The optional dendrogram only orders the rows; it is not another clustering.

Characterization
What makes each cluster special (Section 9).

Stability
Whether the clusters come back when the data changes a little (Section 10).

8.3 Method Details and Cluster Details

Method Details summarises all settings of the run and gives a methods paragraph you can adapt for a paper. **Cluster Details** is a table with one line per data row: name, cluster, plot coordinates, colour and, when available, the membership probability and the row stability. Click a column header to sort.

9 Characterization

The Characterization tab describes each cluster. It is computed on all rows and on the preprocessed values, the same values the clustering used (before any reduction).

9.1 Cluster summary

For each cluster: its size and share of all rows, the *medoid* (the member with the smallest total distance to the other members, i.e. the most typical one), the five members closest to the medoid (the medoid itself first), and its most distinguishing features. Member lists can be copied or downloaded, one name per line, for tools such as g:Profiler or STRING.

9.2 Cluster profiles: median with interquartile range

For each selected cluster, the figure shows its typical profile across the data columns (the line, the median of its members) and, as a shaded band, where the middle half of its members lie (the interquartile range, 25th to 75th percentile). The five largest clusters are shown first; tick others to add them.

Example

A cluster has eight proteins with these values in fraction F3:

$$0.2,\ 0.9,\ 1.1,\ 1.2,\ 1.3,\ 1.4,\ 1.6,\ 3.0$$

The median is 1.25 and the band runs from 1.05 (25th percentile) to 1.45 (75th percentile). The two extreme values 0.2 and 3.0 hardly move these numbers; a mean with standard deviation would be pulled by both.

A narrow band means the members have very similar profiles. A wide band means they differ a lot in that column. Clusters whose lines and bands overlap look alike in those columns.

9.3 Features that distinguish a cluster

For the selected cluster, this figure shows the 15 columns (features) in which its members differ most from all other rows, measured with *Cliff's* δ. Red bars mean higher in the cluster, blue bars lower. The table next to it gives the median in the cluster and the median of the rest; click a feature to see its box plots.

Cliff's δ takes every pair of one value from the cluster and one from the rest and asks which is bigger:

$$\delta = P(\text{cluster value} > \text{rest value}) - P(\text{cluster value} < \text{rest value}).$$

$+1$ means every value in the cluster is higher, -1 every value lower, 0 no difference. $|\delta|$ above about 0.47 is usually called a large effect (about 0.33 medium, 0.15 small). It uses only the order of the values, so it does not depend on units and outliers hardly affect it.

Example: Cliff's δ by hand

In one column, the cluster has the values $3, 4, 5$ and the rest has $1, 2, 3$. There are $3 \times 3 = 9$ pairs:

- 3 against 1, 2, 3: bigger, bigger, tie

- 4 against 1, 2, 3: bigger three times
- 5 against 1, 2, 3: bigger three times

That is 8 pairs bigger, 0 smaller and 1 tie, so $\delta = 8/9 - 0/9 = 0.89$: a large effect, the cluster is clearly higher in this column.

9.4 Feature distribution by cluster

For one chosen column, a box plot of its values in every cluster side by side. The box covers the middle 50 % of the cluster's rows, the solid line is the median, the dashed line the mean, and the whiskers reach the most extreme values within 1.5× the interquartile range. It shows the same numbers as the profiles, seen from the other side: one column across all clusters instead of all columns for a few clusters.

9.5 Differences between clusters per feature (Kruskal–Wallis)

For every column, a Kruskal–Wallis test compares its values across all clusters at once (noise excluded). It is the rank-based version of a one-way ANOVA and does not assume normal data. The table is sorted by the effect size

$$\varepsilon^2 = \frac{H}{n-1},$$

from 0 to 1 (roughly 0.01 small, 0.08 medium, 0.26 and above large). It also shows the test statistic H, the p-value and the q-value (the p-value adjusted for testing many columns, Benjamini–Hochberg).

Example

Three clusters of four rows each have these values in one column:

Cluster 0	1.0, 1.2, 0.9, 1.1
Cluster 1	2.0, 2.2, 1.9, 2.1
Cluster 2	1.0, 1.1, 1.3, 0.8

Cluster 1 is clearly higher. The test gives $H = 7.44$ with $n = 12$ rows, so $\varepsilon^2 = 7.44/11 = 0.68$, a large effect, and $p = 0.024$.

Descriptive only

The clusters were built so that their values differ. Small p-values are therefore expected and are not proof of a real difference; testing on the same data that formed the groups is circular. Use ε^2 to rank the features and do not draw conclusions from the p-values. For real evidence, test the clusters on independent data or check them with the stability analysis.

Cliff's δ and Kruskal–Wallis answer different questions. Cliff's δ compares *one* cluster with the rest and has a direction ("what makes cluster 3 special?"). Kruskal–Wallis looks at all clusters at once without a direction ("which features separate the clusters at all?").

9.6 Outliers and noise

Rows that lie far from their own cluster's median profile, as a robust z-score within the cluster (computed as in the exploratory outlier check, but per cluster), and the list of noise rows. For

HDBSCAN and the mixture models, extra columns show the mean membership, the cluster persistence or the mixture weights.

10 Stability

A clustering algorithm always returns clusters, even for random data. The stability analysis checks whether yours are real: it repeats the clustering on slightly changed data and looks whether the same clusters come back.

What is repeated. 10 random subsamples of 80 % of the rows (adjustable from 3 to 30), and for algorithms with a random seed, 5 runs on all rows with other seeds (0 to 10). All settings stay the same. Each repeated solution is compared with your run on the rows they share.

10.1 Cluster stability (Jaccard)

For each cluster, RapCluster finds the best-matching cluster in every repeated solution and measures their overlap:

$$J = \frac{\text{rows in both}}{\text{rows in either}}.$$

The average over all repeats gives the verdict: **stable** (≥ 0.75), **doubtful** (0.6 to 0.75) or **unstable** (< 0.6).

Example

Cluster 2 contains the rows 1 to 10. In one repeat, its best match contains rows 1 to 8 plus rows 11 and 12. They share 8 rows, and 12 different rows appear in either one, so $J = 8/12 = 0.67$. If the average over all repeats is also around 0.67, cluster 2 is rated doubtful: its core comes back, but its edges keep changing.

10.2 Agreement between solutions (ARI and NMI)

These compare whole clusterings. The *adjusted Rand index* (ARI) looks at every pair of rows and asks whether both clusterings put them together or apart, corrected for chance: 1 means identical, about 0 means no better than random. The *normalized mutual information* (NMI) measures how much one clustering tells about the other, from 0 to 1. Both are shown for each repeat against your run and for the repeats against each other. A high, narrow box plot means the result is reliable; a wide one means it depends on chance.

Example: cluster numbers do not matter

Nine rows, two clusterings:

Row	1	2	3	4	5	6	7	8	9
Run	0	0	0	1	1	1	2	2	2
Repeat	1	1	1	0	0	0	2	2	0

The first six rows are grouped identically; only the numbers 0 and 1 are swapped, which does not count. Row 9 has moved to another cluster. The ARI is 0.64 and the NMI 0.79. Without the move of row 9 the ARI would be exactly 1.

10.3 Row stability and consensus

For every row, the share of the repeats in which it stays in its (matched) cluster. A row that keeps its cluster in 12 of 15 repeats has a row stability of 0.8. The histogram shows all rows, and a table lists the least stable ones, typically rows that sit between two clusters. The *consensus cluster* is the cluster a row was matched to most often. Row stability is also added to Cluster Details, the labels file and the Characterization table.

The analysis is fast: about 1.5 to 2 seconds for 10,000 rows with KMeans or HDBSCAN.

11 Working with many runs

11.1 Run History

All runs of the session, newest first: number, time, dataset, reduction, algorithm, number of clusters, noise, silhouette, runtime, peak memory and status. **View** shows a run and puts its settings back into the form, **Details** lists every setting, and **Labels TSV** and **ZIP** download the results. Failed runs show their error and a **Load settings** button.

11.2 Run Leaderboard

Ranks the runs and names the best one *for the chosen criterion under the chosen constraints*. Another criterion can favour another run; no single score finds the best clustering for every purpose.

- **Rank by:** silhouette, Calinski–Harabasz, DBCV, stability (higher is better), Davies–Bouldin, runtime (lower is better), or a Pareto ranking over several objectives.
- **Constraints:** number of clusters, maximum noise, minimum cluster size, maximum runtime, maximum imbalance (largest cluster divided by smallest) and minimum stability. Excluded runs can be shown with the reason.
- **Recommended badge:** only when the quality (silhouette ≥ 0.25, or a DBCV threshold) *and* the stability (mean ARI ≥ 0.75) both reach their thresholds. Runs without a stability analysis cannot get it.
- **Pin** keeps the current best run in view while you try other settings.

Example: Pareto ranking

Rank by silhouette (higher is better) and runtime (lower is better):

Run	Silhouette	Runtime	Front
A	0.60	10 s	F1
B	0.50	2 s	F1
C	0.40	5 s	F2

B beats C on both objectives, so C is not on the first front. A and B do not beat each other: A has the better silhouette, B is faster. Both are on front F1, and choosing between them is up to you.

Example: constraints

You need between 4 and 8 clusters and at most 10 % noise. A DBSCAN run with the best silhouette but 25 % noise is excluded (shown as "noise 25.0% > 10%" if you show excluded runs), and the best of the remaining runs is named instead.

11.3 Compare Runs

Puts two runs, A and B, side by side: their settings, scatter plots, cluster sizes, profiles, metrics, runtime and warnings; the ARI and NMI between them; the differences in parameters, including preprocessing; and how the rows move between clusters, as a contingency table or a Sankey diagram.

Because cluster numbers have no meaning across runs (cluster 3 in A has nothing to do with cluster 3 in B), clusters are paired by their shared rows. **Color Run B by matched Run A clusters** gives matched clusters the same colour. The rows that changed cluster can be downloaded as a list.

Example

Run A uses KMeans with 6 clusters, run B the same with 7. The Sankey diagram shows that five clusters pass from A to B almost unchanged, while one cluster of A splits into two in B. The list of changed rows contains exactly the members of that cluster.

11.4 Parameter Sweep

Runs the selected algorithm several times with different values of one or two of its parameters. The dataset, preprocessing, reduction and all other parameters stay the same, so the trials are comparable. By default every trial uses all rows.

- For numeric parameters give *from*, *to* and *step*; for options tick the values. *also None* adds the automatic value where a parameter allows it.
- **Max. trials** (up to 30). If there are more combinations, a random subset of combinations is tried, with a fixed seed so it is repeatable. This is a subset of parameter values, not of rows.
- **Screen on a subsample first** (off by default): every trial first runs on a share of the rows (5 to 90 %), and only the best k by the chosen metric are repeated on all rows. The final results are always on the full data.

Example: grid or random subset

DBSCAN with `eps` from 0.3 to 1.0 in steps of 0.1 (8 values) and `min_samples` 3, 5 and 10 gives $8 \times 3 = 24$ combinations. That fits into 30 trials, so all of them run (grid search). With `eps` from 0.1 to 2.0 (20 values) there are 60 combinations, and 30 of them are chosen at random.

The result shows a curve of the metric against the parameter (a heatmap for two parameters), a table of the trials with **View** buttons, failed trials with their error, and the best trial for the chosen metric and constraints. Every finished trial is a normal run with a *sweep* tag, so the Leaderboard, Compare and Stability work on it. The whole sweep is one job under the normal limits; trials that do not start before 85 % of the time limit are marked as skipped.

12 Downloads and reproducing a run

Download All (ZIP) saves everything about a run:

File	Content
`cluster_assignments.tsv`	Cluster of every row, with name, original row number, plot coordinates, and membership and stability when available.
`cluster_sizes.tsv`, `cluster_centroids.tsv`, `cluster_medians.tsv`	Size, mean and median profile per cluster.
`cluster_summary.tsv`, `cluster_profile_bands.tsv`, `distinguishing_features.tsv`, `cliffs_delta_matrix.tsv`, `feature_tests.tsv`, `outliers.tsv`, `noise_points.tsv`, `cluster_members/`	The characterization.
`cluster_stability.tsv`	Stability per cluster, after a stability analysis.
Figures (`.png` and `.svg`)	Scatter plot, cluster sizes, cluster profiles.
`metrics.json`, `config.json`, `run_record.json`, `software_versions.json`, `warnings.txt`	Metrics, settings with seeds, the full run record, package versions and warnings.
`report.html`	A readable report with figures, characterization, stability, settings and a methods text.
`reproduce.py`, `preprocessing.py`	Repeat the run outside the web app.

To repeat a run on your own computer, unpack the ZIP and run:

```
python reproduce.py my_data.tsv
```

It uses the same preprocessing code and the settings and seeds from `config.json` in the same folder (a different settings file can be given as a second argument). With the package versions listed in `software_versions.json` it gives the same clusters as the web app.

13 Running RapCluster yourself

The hosted version is at `https://rapplab.bioanalytik.tu-berlin.de/rapcluster/`. To run your own copy, use Docker:

```
docker compose up --build -d
```

The app is then available at `http://localhost:8088/rapcluster/`. On Linux, macOS or WSL, the script `start.sh` does the same and prints the links:

```
./start.sh

RapCluster is running:
  Local:           http://localhost:8088/rapcluster/
  On your network: http://192.168.2.207:8088/rapcluster/
```

Other computers on the same network can open the "On your network" link. If it does not load for them, allow incoming connections on TCP port 8088 in your firewall. On Windows, in an administrator PowerShell:

```
New-NetFirewallRule -DisplayName "RapCluster 8088" -Direction Inbound -Protocol TCP -
    LocalPort 8088 -Action Allow
```

Memory limits on your own machine

The Docker image is set up for the hosted server with 100 GB of RAM: up to 8 runs at the same time with 10 GB each. On a smaller machine, lower these limits, or a few large runs at once can use up all memory, and the operating system then kills one of them ("The run stopped unexpectedly, most likely because the server ran out of memory"). The rule is: number of runs × memory per run + 3 GB should fit into the RAM. For a laptop with 16 GB, uncomment the `environment` block in `compose.yaml`:

```
environment:
  RAPCLUSTER_JOB_MEMORY_MB: "3072"
  RAPCLUSTER_MAX_CONCURRENT_JOBS: "3"
```

That gives $3 \times 3\,\text{GB} + 3\,\text{GB} = 12\,\text{GB}$.

Note

The server part of RapCluster uses features that only exist on Linux, so it does not run directly on Windows. On Windows, use Docker Desktop or WSL. `start.sh` runs RapCluster on the machine where you start it; it does not deploy anything to the hosted server.

For development, `npm run dev` starts the Flask server on `http://127.0.0.1:5000` and the web page on `http://localhost:3000/rapcluster`. The development server forwards the API calls to Flask, so only port 3000 has to be reachable from other devices.

Glossary

ARI (adjusted Rand index) Agreement of two clusterings over all pairs of rows, corrected for chance. 1 is identical, about 0 is random.

Cliff's δ Effect size between two groups of values, from -1 to $+1$. $|\delta| > 0.47$ is a large effect.

ε^2 (epsilon squared) Effect size of the Kruskal–Wallis test, from 0 to 1.

Imputation Filling missing values.

Interquartile range (IQR) The range between the 25th and the 75th percentile, where the middle half of the values lie.

Jaccard overlap Rows shared by two clusters divided by the rows in either of them.

MAD Median absolute deviation: the median of the distances from the median.

Medoid The cluster member with the smallest total distance to the other members.

NMI (normalized mutual information) How much one clustering tells about another, from 0 to 1.

Noise (-1) Rows that density-based algorithms do not assign to any cluster.

Pareto front The runs that no other run beats on every chosen objective at once.

Robust z-score Distance from the median divided by the scaled MAD. Less affected by outliers than the usual z-score.

Seed (`random_state`) Fixes the random choices of an algorithm, so a run can be repeated exactly.

z-score $(x - \text{mean})/\text{standard deviation}$: how many standard deviations a value lies from the mean.